

Pipes — Built-in & Custom

Angular Learning Series

Category: Angular Core | Difficulty: Beginner-Intermediate | Relevance: High

Modern Angular v17+ Syntax Throughout

Section 0 — Difficulty & Priority Rating

- **Difficulty:** Beginner-Intermediate — the core concept is simple (a class with one method), but the nuance lies in understanding pure vs impure pipes and when each breaks Angular's change detection in ways that are non-obvious until production
- **Angular Relevance:** High — pipes are the primary way to transform display data in templates without polluting component logic; every Angular app uses them, and writing custom ones is a daily skill once built-in pipes fall short

Section 1 — Explanation

The Assembly Line Filter Analogy

A factory assembly line: raw products come in on a conveyor belt, pass through a filter/stamping machine, and come out transformed — painted, resized, relabelled. The machine does not change the original product sitting in the warehouse; it only changes how the product looks as it passes through for display.

A pipe is that machine. Data flows in from your component, passes through `transform()`, and comes out formatted for the template. The original data in your component class is untouched.

Angular's built-in pipes (`DatePipe`, `CurrencyPipe`, `UpperCasePipe`) are pre-built machines on the factory floor. A custom pipe is a machine you build yourself when none of the pre-built ones do the job.

Technical Explanation

A custom pipe is a TypeScript class decorated with `@Pipe({ name: '...' })` that implements the `PipeTransform` interface. That interface requires exactly one method: `transform(value, ...args)`. In a template, the pipe is invoked with the `|` operator, and any arguments after the colon are forwarded to `transform()` as additional parameters:

```
{{ expression | pipeName : arg1 : arg2 }}
```

Pure vs Impure — the critical distinction

	Pure Pipe	Impure Pipe
-----	-----	-----
pure flag	true (default)	false
When <code>transform()</code> runs	Only when input reference changes	Every change detection cycle
Performance	Fast — Angular memoises result	Slow — runs on every tick
Safe for	Primitives, immutable objects	Mutable arrays/objects, async data
Risk	Misses mutations inside same reference	Can tank performance if heavy

The golden rule: A pure pipe sees a new array only if you replace the array reference. Pushing into the same array is invisible to a pure pipe. This is the single most common pipe bug in Angular.

Section 2 — Modern Angular Pattern (v17+)

All pipes in v17+ are declared with `standalone: true` and imported directly into each consuming component's imports array — no `NgModule` declarations needed.

```
@Pipe({ name: 'truncate', standalone: true, pure: true })
```

```

export class TruncatePipe implements PipeTransform {
  transform(value: string | null | undefined, maxLen = 50, suffix = '...'): string {
    if (value == null) return '';
    if (value.length <= maxLen) return value;
    return value.slice(0, maxLen).trimEnd() + suffix;
  }
}

// Consuming component – pipe goes in imports[], not declarations[]
@Component({
  standalone: true,
  imports: [TruncatePipe],
  template: `<p>{{ article().body | truncate : 120 : ' [read more]' }}</p>`
})
export class ArticleCardComponent {
  article = input.required<{ title: string; body: string }>();
}

```

Built-in pipes commonly used

DatePipe, CurrencyPipe, DecimalPipe, UpperCasePipe, LowerCasePipe, TitleCasePipe, AsyncPipe, JsonPipe, KeyValuePipe, SlicePipe. All importable directly from @angular/common.

Key modern rule — immutable updates required for pure filter/sort pipes

```

// WRONG – pure pipe never sees a new reference
this.tasks.push(newTask);

// CORRECT – new reference triggers pure pipe re-evaluation
this.tasks = [...this.tasks, newTask];

```

Section 3 — Code Example

```

// Plain TS – the core concept in isolation
class TruncatePipe {
  transform(value: string, maxLength = 50, ellipsis = '...'): string {
    if (!value) return '';
    if (value.length <= maxLength) return value;
    return value.slice(0, maxLength) + ellipsis;
  }
}

const pipe = new TruncatePipe();
console.log(pipe.transform('Hello Angular World', 10)); // "Hello Angu..."

// truncate.pipe.ts – full working standalone pipe
import { Pipe, PipeTransform } from '@angular/core';

@Pipe({ name: 'truncate', standalone: true, pure: true })
export class TruncatePipe implements PipeTransform {
  transform(value: string | null | undefined, maxLen = 50, suffix = '...'): string {
    if (value == null) return '';
    if (value.length <= maxLen) return value;
    return value.slice(0, maxLen).trimEnd() + suffix;
  }
}

// filter-by-status.pipe.ts – pipe with multiple arguments

```

```

import { Pipe, PipeTransform } from '@angular/core';

interface Task { id: number; title: string; status: 'open' | 'done' | 'archived'; }

@Pipe({ name: 'filterByStatus', standalone: true, pure: true })
export class FilterByStatusPipe implements PipeTransform {
  transform(tasks: Task[] | null, status: string): Task[] {
    if (!tasks) return []; // guard – async data may arrive as null
    if (!status) return tasks;
    return tasks.filter(t => t.status === status);
  }
}

// task-list.component.ts
@Component({
  standalone: true,
  imports: [FilterByStatusPipe],
  template: `
    @for (task of (tasks | filterByStatus : activeFilter); track task.id) {
      <p>{{ task.title }}</p>
    }
    <button (click)="setFilter('open')">Open</button>
    <button (click)="setFilter('done')">Done</button>
  `
})
export class TaskListComponent {
  tasks: Task[] = [];
  activeFilter = 'open';
  setFilter(status: string) {
    this.activeFilter = status;
    // activeFilter is a string primitive – change triggers pure pipe re-run
  }
}

// format-currency.pipe.ts – real-world production pipe
@Pipe({ name: 'formatCurrency', standalone: true, pure: true })
export class FormatCurrencyPipe implements PipeTransform {
  transform(value: number | null, currency = 'USD', locale = 'en-US'): string {
    if (value == null) return '-'; // graceful dash for missing data
    return new Intl.NumberFormat(locale, {
      style: 'currency',
      currency,
      minimumFractionDigits: 2
    }).format(value);
  }
}

// Used across multiple components without code duplication:
// <td>{{ trade.notional | formatCurrency : 'GBP' : 'en-GB' }}</td>
// <span>{{ account.balance | formatCurrency }}</span>
// <p>{{ position.pnl | formatCurrency : 'EUR' : 'de-DE' }}</p>

```

Section 4 — Before & After Refactor

```
// BEFORE – transformation logic polluting the component
@Component({
  template: `
    <p>{{ truncatedBody }}</p>
    <span>{{ formattedPrice }}</span>
  `
})
export class ProductComponent implements OnInit {
  product!: Product;
  truncatedBody = '';
  formattedPrice = '';

  ngOnInit() {
    this.productService.get(this.id).subscribe(p => {
      this.product = p;
      // Runs once – goes stale if product updates later
      this.truncatedBody = p.description.slice(0, 100) + '...';
      this.formattedPrice = '$' + p.price.toFixed(2);
    });
  }
}

// AFTER – clean component, reusable pipes, signal input
@Component({
  standalone: true,
  imports: [TruncatePipe, CurrencyPipe],
  template: `
    <p>{{ product().description | truncate : 100 }}</p>
    <span>{{ product().price | currency : 'USD' }}</span>
  `
})
export class ProductComponent {
  product = input.required<Product>();
  // Zero transformation code.
  // If product() updates, pipes re-run automatically.
}
```

Why it matters: Transformation ran once at load time in the before — if the product updated, the values went stale. The after version has pipes reusable across the entire app, Angular's pure pipe memoisation avoids wasted computation, and the component can go OnPush without any extra work.

Section 5 — Common Mistakes

Mistake 1 — Using a pure pipe on a mutated array

```
// WRONG – mutates same reference – pure pipe never re-runs
this.tasks.push(task);

// CORRECT – new reference forces pure pipe re-evaluation
this.tasks = [...this.tasks, task];
```

The filtered/sorted pipe result silently does not update. Users see stale data with no console error — one of the hardest bugs to spot.

Mistake 2 — Making every array pipe impure "to be safe"

After hitting the mutation bug, the reflex fix is pure: false. Fix the mutation instead. An impure pipe on a list of 1,000 items runs filter() on every keypress, scroll, mouse move, and timer tick — direct route to frame drops and sluggish UIs.

Mistake 3 — Forgetting to import the pipe in the component

NG8004: Pipe 'truncate' is not known — the pipe class has not been added to imports: [] of the standalone component. In standalone Angular, every consumer must import the pipe directly. Build-time error — blocks the build until resolved.

Section 6 — Do's and Don'ts

- DO: Keep pure: true (the default) for pipes transforming primitives or immutable inputs
- DON'T: Set pure: false as a quick fix for stale data — diagnose the mutation instead
- DO: Return a new array/object reference from transform() when input is a collection
- DON'T: Mutate the value parameter inside transform() — pipes must be side-effect free
- DO: Add standalone: true and import the pipe directly into each consumer's imports array
- DON'T: Declare pipes in an NgModule then wonder why template compilation fails
- DO: Handle null and undefined on the first line — async data arrives late
- DON'T: Assume value will always be the expected type — API data is always nullable on first load
- DO: Keep transform() synchronous
- DON'T: Inject HttpClient or perform async operations inside a pipe — use a service + async pipe instead
- DO: Write a unit test for every custom pipe — transform() is a pure function and tests are trivial
- DON'T: Skip pipe tests — pipes encode real business formatting logic

Section 7 — Interview Prep

Question 1

Question: "What is the difference between a pure and an impure pipe in Angular, and when would you use each?"

Strong answer: Pure pipes are called only when the input reference changes — Angular memoises the last result. Impure pipes run on every change detection cycle. Pure pipes are the default and correct choice for most formatting/filtering scenarios. Impure pipes are appropriate when the output depends on external mutable state not reflected in the input value — current time, user locale from a service, translated strings. The performance cost of impure pipes: runs on every mousemove, keydown, timer tick. Pure pipes with array inputs require immutable updates — push() on the same reference means the pipe never re-runs.

Weak answer misses: That pure pipes require immutable array updates — many candidates know the theory but miss the practical implication.

Level: Mid

Question 2

Question: "Why might a filter pipe applied to an array stop working after items are added?"

Strong answer: Angular's pure pipe caches results based on reference equality. `Array.push()` mutates the existing array in place — the reference does not change. Angular's pure pipe mechanism sees the same reference and returns the cached stale result. Fix: create a new array with spread (`[...arr, newItem]`) so the pipe sees a new reference. Alternative: set `pure: false`, but explain the performance trade-off — impure pipes run on every CD cycle.

Weak answer misses: The word "reference" — candidates often say "the array doesn't update" without explaining the reference-equality mechanism.

Level: Mid

Question 3

Question: "How would you build a currency-formatting pipe that handles null values and is usable across multiple standalone components?"

Strong answer: Implement `PipeTransform`, decorate with `@Pipe({ name: '...', standalone: true, pure: true })`. First line of `transform()` guards for null/undefined — returns a placeholder like '—'. Use `Intl.NumberFormat` for formatting. Mark as `standalone: true` so it can be imported directly into any component's imports array. The pipe goes in imports: `[]` of consuming components, not declarations: `[]`.

Weak answer misses: Handling null inputs and the `standalone: true + imports` pattern — many candidates still think in `NgModule` terms.

Level: Mid-Senior

Section 8 — Quick Reference Card

3-line definition

A pipe is a `transform(value, ...args)` method Angular calls in the template to convert raw data into a display-ready format. Pure pipes (default) are memoised — `transform()` only runs when the input reference changes. Impure pipes run on every change detection cycle — use sparingly.

Syntax at a glance

```
@Pipe({ name: 'myPipe', standalone: true, pure: true })
export class MyPipe implements PipeTransform {
  transform(value: T | null, arg1: string): string {
    if (value == null) return '';
    // ...transform logic
  }
}
// Template: {{ value | myPipe : 'arg1' }}
// Component: imports: [MyPipe]
```

Pure vs Impure — when to use which

Use Pure when...	Use Impure when...
-----	-----

Input is a primitive	Output depends on external mutable state
Input array is replaced on update	Input array is mutated in place
Formatting dates, numbers, strings	Building a translate/i18n pipe
Filtering immutable collections	Pipe needs service state not in the input

The one rule that matters most

If a pure pipe stops updating after you add items to an array, you are mutating the array in place
— replace it with a new reference using spread syntax:

```
this.items = [...this.items, newItem];
```